

Efficient Provable Data Possession with Batch Verification Across Multiple Files

Abstract—

Index Terms—Polynomial Commitment, Vector Commitment, Data Management

I. INTRODUCTION

a) Unified Challenge Design via Dual-Purpose r_i :

A salient innovation of our construction lies in the dual-purpose utilization of the per-file challenge point r_i . Unlike conventional batch verification schemes that require disjoint random values for polynomial evaluation and linear aggregation, our protocol ingeniously unifies these roles by employing r_i simultaneously as the evaluation point for the per-file quotient polynomial $g_i(X) = (f_i(X) - y_i)/(X - r_i)$ and as the batching coefficient in the cross-file quotient commitment $C_{qr} = g^{\sum_{i=1}^m r_i g_i(\tau)}$. This unified challenge design eliminates the overhead of sampling and transmitting auxiliary aggregation scalars, directly reducing the challenge payload and streamlining the Fiat-Shamir transcript construction. More importantly, it induces a precise algebraic cancellation in the quotient-consistency equation $e(A \cdot C_{qr}, h) \stackrel{?}{=} e(C_q, h^\tau)$, which compresses m independent quotient checks into a single constant-size pairing verification. Despite this structural reuse, cryptographic soundness is rigorously maintained: since r_i is uniformly sampled by the verifier (or derived via Fiat-Shamir) after the homomorphic tags are irrevocably fixed, it functions as a fresh, unpredictable random challenge that thwarts adaptive quotient forgery under the Schwartz-Zippel lemma. Consequently, our approach achieves optimal challenge efficiency and transcript compactness without compromising the knowledge soundness of the underlying KZG commitments, marking a distinct advancement over prior multi-file PDP frameworks that rely on disjoint challenge spaces for evaluation and aggregation.

II. PRELIMINARIES

A. Notations

For clarity and reference, we summarize the primary mathematical symbols, cryptographic parameters, and protocol-specific variables used throughout the paper in Table I.

B. Basics of Polynomials

Let $\mathbb{F}_p$ denote a finite field of prime order p , where p is determined by a security parameter λ . For a positive integer d , let $\mathbb{F}_p^{<d}[X]$ denote the vector space of univariate polynomials over $\mathbb{F}_p$ with degree strictly less than d . Polynomial indeterminates are denoted by uppercase letters (e.g., X), while field elements and coefficients use lowercase letters (e.g., $x, z, r \in \mathbb{F}_p$).

Table I: Summary of Notation and System Parameters

Symbol	Description
λ	Security parameter.
p	Prime order of the finite field $\mathbb{F}_p$ and bilinear groups.
$[m]$	Index set $\{1, 2, \dots, m\}$.
$\mathbb{F}_p^{<d}[X]$	Vector space of univariate polynomials over $\mathbb{F}_p$ with degree strictly less than d .
g, h	Public generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively.
τ	Secret trapdoor used for Structured Reference String (SRS) generation.
SRS	Public parameters $(g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$ supporting degrees $< s$.
H_1	Hash function for Merkle Tree construction.
H_2	Fiat-Shamir random oracle hash functions.
m, n, s, c	Number of files, chunks per file, sectors per chunk, and challenged chunks per file.
i, j, k	Indices for files ($i \in [m]$), chunks ($j \in [n]$), and sectors ($0 \leq k < s$).
$m_{i,j,k}$	k -th sector value of chunk j in file i .
$f_{i,j}(X)$	Polynomial encoding of chunk j in file i : $f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k$.
$\sigma_{i,j}$	KZG commitment (homomorphic tag) for $f_{i,j}(X)$: $\sigma_{i,j} = g^{f_{i,j}(\tau)}$.
$L_{i,j}$	Leaf node of Merkle tree: $L_{i,j} = H_1(i \ j \ \sigma_{i,j})$.
Γ_i	Root hash of the Merkle tree for file i .
J	Challenged index set, $J \subseteq [n]$ with $ J = c$.
v_j	Short for $v_{i,j}$, random coefficient for challenged chunk j of file i .
r_i	Random evaluation point for file i .
$f_i(X)$	Aggregated polynomial for file i : $f_i(X) = \sum_{j \in J} v_j f_{i,j}(X)$.
y_i	Evaluation of aggregated polynomial: $y_i = f_i(r_i)$.
b_i	Aggregated tag for file i : $b_i = \prod_{j \in J} \sigma_{i,j}^{v_j} = g^{f_i(\tau)}$.
$w_i(X)$	Per-file quotient polynomial: $w_i(X) = (f_i(X) - y_i)/(X - r_i)$.
$q(X)$	Aggregated quotient polynomial: $q(X) = \sum_{i=1}^m w_i(X)$.
C_q, C_{qr}	Quotient commitments: $C_q = g^{q(\tau)}$, $C_{qr} = g^{\sum_{i=1}^m r_i w_i(\tau)}$.
α_i	Fiat-Shamir derived folding coefficient for file i .
$Q(X)$	Folded polynomial: $Q(X) = q(X) + \sum_{i=1}^m \alpha_i f_i(X)$.
C_Q	Folded commitment: $C_Q = C_q \cdot \prod_{i=1}^m b_i^{\alpha_i}$.
z	Random evaluation point for folded opening (Fiat-Shamir derived).
V_Q	Folded evaluation: $V_Q = Q(z)$.
μ_i, η	Fresh additive masking scalars for per-file and global privacy.
$\tilde{y}_i, \tilde{V}_Q$	Masked evaluations: $\tilde{y}_i = y_i + \mu_i$, $\tilde{V}_Q = V_Q + \eta$.
R_i, R_Q	Masking commitments: $R_i = g^{\mu_i}$, $R_Q = g^\eta$.
π^{batch}	Batch opening proof: $g^{(Q(\tau) - V_Q)/(\tau - z)}$.
chal	Challenge tuple $(J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m)$.
π	Complete audit proof.

a) *Quotient Polynomial*: Given a polynomial $f(X) \in \mathbb{F}_p^{<d}[X]$ and an evaluation point $z \in \mathbb{F}_p$, the *quotient polynomial* with respect to z is defined as $w(X) := \frac{f(X) - y}{X - z}$. Whenever $f(z) = y$ for some $y \in \mathbb{F}_p$, the numerator vanishes at $X = z$, ensuring that $w(X)$ is a well-defined polynomial in $\mathbb{F}_p^{<d-1}[X]$. Given polynomial $f(x)$, quotient polynomial $w(x)$ and a fixed evaluation point τ , we have $(\tau - z) \cdot w(\tau) = f(\tau) - y$. Rearranging the equation yields $\tau \cdot w(\tau) = f(\tau) - y + z \cdot w(\tau)$. When considering aggregating multiple polynomials $\{f_i\}$ and $\{w_i\}$, the protocol sums all

equations over $i \in [m]$ as:

$$\tau \sum_{i=1}^m w_i(\tau) = \sum_{i=1}^m (f_i(\tau) - y_i) + \sum_{i=1}^m z_i \cdot w_i(\tau). \quad (1)$$

b) *Schwartz-Zippel Lemma*: The Schwartz-Zippel lemma is a fundamental probabilistic tool for bounding the likelihood that a non-zero polynomial evaluates to zero at a randomly chosen point. It serves as the mathematical cornerstone for the soundness guarantees of polynomial commitment schemes and probabilistic proof systems.

Lemma 1 (Schwartz-Zippel Lemma [?], [?]). *Let $\mathbb{F}$ be a field, and let $P(X) \in \mathbb{F}[X]$ be a non-zero polynomial of degree $d \geq 0$. Let $S \subseteq \mathbb{F}$ be a finite subset. If r is chosen uniformly at random from S , then*

$$\Pr_{r \leftarrow S}[P(r) = 0] \leq \frac{d}{|S|}.$$

C. KZG Polynomial Commitment

We employ a KZG-type polynomial commitment scheme over a bilinear group tuple $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ of prime order p . Here, $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is an efficiently computable, non-degenerate bilinear map. Let $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$ be publicly known generators.

A trusted setup generates a secret trapdoor $\tau \in \mathbb{F}_p$ and publishes a structured reference string (SRS) supporting polynomials up to degree $s - 1$:

$$\text{SRS} = (g, g^\tau, g^{\tau^2}, \dots, g^{\tau^{s-1}}, h, h^\tau).$$

For a polynomial $f(X) = \sum_{k=0}^{s-1} a_k X^k \in \mathbb{F}_p^{<s}[X]$, its commitment is computed in the exponent as

$$C_f := g^{f(\tau)} = \prod_{k=0}^{s-1} (g^{\tau^k})^{a_k} \in \mathbb{G}_1.$$

To prove an evaluation $f(z) = v$, the prover computes the quotient commitment $\pi := g^{w(\tau)}$ where $w(X) = (f(X) - v)/(X - z)$. The verifier checks the pairing equation

$$e(C_f \cdot g^{-v}, h) \stackrel{?}{=} e(\pi, h^{\tau-z}),$$

which holds if and only if $f(\tau) - v = (\tau - z)w(\tau)$ and $w(X)$ is the quotient polynomial. Under the q -SDH assumption [?], the scheme is evaluation-binding and knowledge-sound.

D. Homomorphic Hash Function

A hash algorithm is a mathematical function that transforms input data of arbitrary size into a fixed-size output, called a hash value, which is designed to be unique, fast to compute, and practically irreversible. Define the hash function $H : \mathbb{F}_p \rightarrow \mathbb{G}_1$ as: $\sigma := H(m) = g^m$.

- *One-wayness*: It is easy to compute the hash value σ , but computationally infeasible to recover the original z input from the hash due to the discrete logarithm hard problem.
- *Homomorphism*: The hash function H is preimage-resistant and preserves linear combinations, namely, for any integer t and any $\{m_i\}_{i=1}^t$ and $\{v_i\}_{i=1}^t \subseteq \mathbb{F}_p$, $H(\sum_{i=1}^t v_i m_i) = g^{\sum_{i=1}^t v_i m_i} = \prod_{i=1}^t g^{v_i m_i} = \prod_{i=1}^t H(m_i)^{v_i}$ holds.

E. Merkle Tree

A Merkle tree is a binary hash-based commitment structure that provides authenticated membership proofs for a set of values. Let $H_1 : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be a collision-resistant cryptographic hash function. For a sequence of n data items $\{d_j\}_{j=1}^n$, the leaves are constructed as indexed hashes: $L_j := H_1(j \parallel d_j)$, $j \in [n]$. Internal nodes are recursively computed as $N_k = H_1(N_{2k} \parallel N_{2k+1})$ until a single root hash R is obtained. The root R uniquely commits to the entire dataset under the collision resistance of H_1 .

To prove that a value d_j belongs to the committed set, the prover provides an *authentication path* consisting of the sibling hashes along the route from leaf L_j to the root. The verifier recomputes the root from the path and checks equality with the trusted root R .

III. SYSTEM MODEL AND SECURITY MODEL

A. System Model

a) *System Entities*: We consider a batch-verifiable Provable Data Possession (PDP) architecture comprising three entities: Data Owner, a Cloud Server and Verifier.

- **Data Owner (DO)**: The DO outsources a batch of m files to the CS for long-term storage. To ensure data integrity without maintaining local copies, the DO delegates periodic auditing to the V.
- **Cloud Server (CS)**: The CS is modeled as untrusted and computationally bounded: it may delete, corrupt, or selectively withhold files or chunks to save storage or bandwidth.
- **Verifier (V)**: The V is honest-but-curious. It executes the auditing protocol faithfully but may attempt to infer sensitive information from audit transcripts.

b) *File Representation*: A file is divided into multiple chunks, and each **chunk** can be represented through polynomial coefficients. Each coefficient is referred to as a **sector**. Let m denote the number of files in a single audit batch, n the number of chunks per file, s the number of sectors per chunk. Namely, each chunk is parsed into s field elements from $\mathbb{F}_p$ and algebraically encoded as a polynomial of degree at most $s - 1$:

$$f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k, \quad i \in [m], j \in [n],$$

where $m_{i,j,k}$ represents the k -th sector of the j -th chunk in file i . A **tag** is used to represent the j -th chunk through a KZG polynomial commitment. i.e., $\sigma_{i,j} = H(f_{i,j}(\tau)) = g^{f_{i,j}(\tau)} \in \mathbb{G}_1$, where τ is the trapdoor from KZG polynomial setup. Further, tags of each file are organized into a **Merkle Tree** to guarantee correct indexing and membership.

c) *Protocol Workflow*: The provable data possession scheme for multiple files operates through five polynomial-time algorithms:

- $\text{Setup}(\lambda) \rightarrow \text{pp}$: Generates public parameters pp , including bilinear group tuples $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$, a structured reference string (SRS) for degree $s - 1$, and cryptographic

hash functions H_1 (for Merkle Tree) and H_2, H_2 (for Fiat-Shamir transcript hashing).

- $\text{Store}(\{F_i\}_{i=1}^m) \rightarrow (\text{data}, \text{meta})$: The DO encodes each chunk into $f_{i,j}(X)$, computes tags $\sigma_{i,j}$, constructs the Merkle Tree to obtain the root hash Γ_i for each file, and outsources $\{F_i, \{\sigma_{i,j}\}, \text{Merkle trees}\}$ to the CS. The DO securely records $\{\Gamma_i\}_{i=1}^m$ and shares them with V.
- $\text{Chal}(m, n, c) \rightarrow \text{chal}$: The verifier samples a shared challenge index set $J \subseteq [n]$ with $|J| = c$, random aggregation coefficients $\{v_{i,j}\}_{i \in [m], j \in J} \leftarrow \mathbb{F}_p$, and per-file evaluation points $\{r_i\}_{i=1}^m \leftarrow \mathbb{F}_p$. For brevity, we omit the index i in $v_{i,j}$ and simply write v_j to denote $v_{i,j}$. The challenge is $\text{chal} = (J, \{v_j\}, \{r_i\}, \text{nonce})$, where nonce is a random value to resist replay attack.
- $\text{ProofGen}(\text{chal}, \text{data}, \text{meta}) \rightarrow \pi$: The CS aggregates challenged chunks J and outputs the corresponding non-interactive proofs π .
- $\text{Verify}(\text{chal}, \pi, \{\Gamma_i\}, \text{pp}) \rightarrow \{accept, reject\}$: V validates the integrity of all files $[m]$ via the proofs π .

B. Security Model

The security of the proposed batch-PDP scheme is formalized through three core properties: correctness, soundness, and privacy.

Definition 1 (Correctness). *If all parties (DO, CS, and V) execute the protocol honestly according to the specified algorithms, the Verify algorithm outputs accept with probability 1.*

Definition 2 (Soundness). *Let $\mathcal{A}$ be a probabilistic polynomial-time (PPT) adversary controlling the CS. $\mathcal{A}$ wins the soundness game if it can produce a proof π that causes Verify to output accept for a challenge involving corrupted, modified, or unavailable data chunks with non-negligible probability. The scheme is sound if $\Pr[\mathcal{A} \text{ wins}] \leq \text{negl}(\lambda)$, assuming:*

- 1) H_1 is collision-resistant;
- 2) The KZG polynomial commitment scheme is evaluation-binding and knowledge-sound (e.g., under the q -SDH or q -DLOG assumption);
- 3) The Fiat-Shamir hash functions H_2 is modeled as random oracle.

Definition 3 (Privacy against Honest-but-Curious Verifier). *Given the public audit transcript and the generated proof π , a PPT verifier cannot recover the challenged evaluations $\{y_i = f_i(r_i)\}_{i=1}^m$ or the underlying raw data sectors with non-negligible advantage.*

IV. PROPOSED SCHEME

A. Construction

a) *Setup*: The Setup algorithm initializes the cryptographic environment and publishes the system-wide public parameters pp . It first generates a bilinear group tuple $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ of prime order p , selects public generators $g \in \mathbb{G}_1$ and $h \in \mathbb{G}_2$, and samples a secret trapdoor $\tau \in \mathbb{F}_p$. Using τ , the algorithm constructs the

structured reference string (SRS) $(g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$, which supports polynomial commitments of degree strictly less than s . Cryptographic hash functions are fixed: H for homomorphic hashing, H_1 for Merkle Tree construction, and H_2 for Fiat-Shamir transcript binding. Finally, $\text{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h, \text{SRS}, H, H_1, H_2)$ is published. Note that each DO has its own trapdoor τ to enable batch of polynomials and guarantee privacy of DO's files.

Algorithm 1 Setup

Require: security parameter λ

Ensure: public parameters pp

- 1: Generate $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h)$ and $\text{SRS} = (g, g^\tau, \dots, g^{\tau^{s-1}}, h, h^\tau)$.
- 2: Fix cryptographic hashes H, H_1, H_2 .
- 3: Publish

$$\text{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p, g, h, \text{SRS}, H, H_1, H_2).$$

b) *Files Storing*: The Store algorithm handles data pre-processing, cryptographic tagging, and metadata generation for outsourcing. For each file F_i ($i \in [m]$), the Data Owner parses every chunk $j \in [n]$ into s field elements and algebraically encodes them as a polynomial $f_{i,j}(X) = \sum_{k=0}^{s-1} m_{i,j,k} X^k$. A KZG homomorphic tag $\sigma_{i,j} = g^{f_{i,j}(\tau)} \in \mathbb{G}_1$ is computed for each chunk, providing a compact cryptographic binding to the underlying data sectors. These tags are organized as leaves in a Merkle Tree, where each leaf is hashed as $L_{i,j} = H_1(i \parallel j \parallel \sigma_{i,j})$ to cryptographically bind tags to their file and chunk indices. The tree root Γ_i is securely retained by the DO (and later shared with the Verifier), while the raw data, tags, and Merkle authentication nodes are outsourced to the CS. This structure guarantees both data integrity and index authenticity without requiring the DO to maintain local copies.

Algorithm 2 Store

Require: files $\{F_i\}_{i=1}^m$

Ensure: outsourced data, metadata

- 1: **for** $i = 1$ to m **do**
 - 2: **for** $j = 1$ to n **do**
 - 3: Parse chunk j into s sectors and form $f_{i,j}(X)$.
 - 4: Compute tag $\sigma_{i,j} = H(f_{i,j}(\tau)) = g^{f_{i,j}(\tau)}$.
 - 5: $L_{i,j} = H_1(i \parallel j \parallel \sigma_{i,j})$
 - 6: **end for**
 - 7: Build the i th Merkle Tree over leaves $\{L_{i,j}\}_{j=1}^n$.
 - 8: Obtain root Γ_i .
 - 9: **end for**
 - 10: DO sends $(F_i, \{\sigma_{i,j}\})$ to CS and records $\{\Gamma_i\}$ for V.
-

c) *Challenge Generation*: The Chal algorithm initiates the audit by sampling a challenge tuple $\text{chal} = (J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m)$, where v_j is short for $v_{i,j}$. An index set $J \subseteq [n]$ of size c for each file is selected to query the chunk positions. Random coefficients $\{v_j\}$ are drawn uniformly from $\mathbb{F}_p$ to enable random linear aggregation of challenged chunks; this masks individual chunk values, prevents targeted

corruption, and enables homomorphic tag recombination. Additionally, independent per-file evaluation points $\{r_i\}$ are sampled to support file-specific quotient consistency checks. The challenge is transmitted to the CS alongside a fresh nonce to prevent replay or precomputation attacks.

Algorithm 3 Chal

Require: parameters (m, n, c)

Ensure: challenge chal

- 1: Sample one shared challenged index set $J \subseteq [n]$, $|J| = c$.
 - 2: Sample random $\{v_j \xleftarrow{\$} \mathbb{F}_p\}_{j \in J}$.
 - 3: Sample random $\{r_i \xleftarrow{\$} \mathbb{F}_p\}_{i=1}^m$
 - 4: Set

$$\text{chal} = (J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m, \text{nonce}).$$
-

d) *Proof Generation:* Upon receiving Chal, the ProofGen algorithm computes a compact, cross-file aggregated proof. For each file i , the challenged chunks are linearly combined into an aggregated polynomial $f_i(X) = \sum_{j \in J} v_j f_{i,j}(X)$, with its corresponding aggregated tag $b_i = g^{f_i(\tau)}$. To preserve data privacy against an honest-but-curious verifier, the exact evaluation $y_i = f_i(r_i)$ is additively masked as $\tilde{y}_i = y_i + \mu_i$, and the mask is committed as $R_i = g^{\mu_i}$. The CS then constructs per-file quotient polynomials $w_i(X) = \frac{f_i(X) - y_i}{X - r_i}$ and aggregates them into a global quotient $q(X) = \sum_{i=1}^m w_i(X)$, producing commitments $C_q = g^{q(\tau)}$ for polynomial $q(x)$ and $C_{qr} = g^{\sum_{i=1}^m r_i w_i(\tau)}$ for linear combination of polynomials $w_i(x)$. Crucially, the Fiat-Shamir heuristic is applied to derive folding coefficients α_i after (C_q, C_{qr}) are fixed in the transcript, preventing adaptive adversarial manipulation of the quotient layer. A final folded polynomial $Q(X) = q(X) + \sum_{i=1}^m \alpha_i f_i(X)$ is opened at a Fiat-Shamir-derived point z , yielding a single batch opening proof $\pi_{\text{batch}} = g^{\frac{Q(\tau) - V_Q}{\tau - z}}$ alongside a masked global evaluation $\tilde{V}_Q$ and commitment R_Q . The complete proof π is returned to the verifier V.

e) *Verification:* The Verify algorithm validates the proof through a sequence of deterministic checks that require only a constant number of pairings, independent of the batch size m . First, the Merkle tree indices J is verified against each stored root Γ_i , and the tag recombination $\hat{b}_i = \prod_{j \in J} \sigma_{i,j}^{v_j}$ is checked against the claimed b_i to ensure index authenticity and tag consistency. Second, the quotient-consistency relation is enforced via the pairing equation $e(A \cdot C_{qr}, h) \stackrel{?}{=} e(C_q, h^\tau)$, where $A = \prod_{i=1}^m (b_i \cdot R_i \cdot g^{-\tilde{y}_i})$. Third, the verifier independently derives $\{\alpha_i\}$ using the same Fiat-Shamir transcript, reconstructs the folded commitment C_Q and evaluation point z , and verifies the final opening check $e(C_Q \cdot R_Q \cdot g^{-\tilde{V}_Q} \cdot \pi_{\text{batch}}^z, h) \stackrel{?}{=} e(\pi_{\text{batch}}, h^\tau)$. Successful validation guarantees that all challenged data across m files are intact, correctly indexed, and unmodified, while the additive masks $\{\mu_i, \eta\}$ ensure that no raw data or exact polynomial evaluations are leaked during the audit.

B. Security Analysis

We formally establish the correctness, soundness, and privacy of the proposed batch-PDP scheme. The security analysis

Algorithm 4 ProofGen

Require: challenge chal = $(J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$

Ensure: proof π

- 1: **for** $i = 1$ to m **do**
 - 2: $f_i(X) \leftarrow \sum_{j \in J} v_j f_{i,j}(X)$
 - 3: $b_i \leftarrow \prod_{j \in J} \sigma_{i,j}^{v_j} = g^{f_i(\tau)}$
 - 4: $y_i \leftarrow f_i(r_i)$
 - 5: sample $\mu_i \leftarrow \mathbb{F}_p$, set $\tilde{y}_i \leftarrow y_i + \mu_i$, $R_i \leftarrow g^{\mu_i}$
 - 6: build authentication path Auth_i for the i th Merkle tree
 - 7: $w_i(X) \leftarrow \frac{f_i(X) - y_i}{X - r_i}$
 - 8: **end for**
 - 9: $q(X) \leftarrow \sum_{i=1}^m w_i(X)$
 - 10: $C_q \leftarrow g^{q(\tau)}$
 - 11: $C_{qr} \leftarrow \prod_{i=1}^m (g^{w_i(\tau)})^{r_i} = g^{\sum_{i=1}^m r_i w_i(\tau)}$
 - 12: **for** $i = 1$ to m **do**
 - 13: $\alpha_i \leftarrow H_2(\text{chal}, \{\Gamma_i\}, \{b_i, \tilde{y}_i, R_i\}, C_q, C_{qr}, i)$
 - 14: **end for**
 - 15: $Q(X) \leftarrow q(X) + \sum_{i=1}^m \alpha_i f_i(X)$
 - 16: $C_Q \leftarrow C_q \cdot \prod_{i=1}^m b_i^{\alpha_i}$
 - 17: $z \leftarrow H_2(\text{chal}, \{\Gamma_i\}, \{b_i, \tilde{y}_i, R_i\}, C_q, C_{qr}, C_Q)$
 - 18: $V_Q \leftarrow Q(z)$
 - 19: sample $\eta \leftarrow \mathbb{F}_p$, set $\tilde{V}_Q \leftarrow V_Q + \eta$, $R_Q \leftarrow g^\eta$
 - 20: $\pi_{\text{batch}} \leftarrow g^{\frac{Q(\tau) - V_Q}{\tau - z}}$
 - 21: **return**

$$\pi = (\{T_i, \text{Auth}_i, b_i, \tilde{y}_i, R_i\}_{i=1}^m, C_q, C_{qr}, \tilde{V}_Q, R_Q, \pi_{\text{batch}})$$
-

relies on the following standard cryptographic assumptions:

Assumption 1 (Collision-Resistant Hashing). *The hash functions H_1 (for Merkle tree construction) and H_2 (for Fiat-Shamir transcript binding) are collision-resistant. No PPT adversary can find distinct inputs $x \neq x'$ such that $H_1(x) = H_1(x')$ or $H_2(x) = H_2(x')$ with non-negligible probability.*

Assumption 2 (KZG Knowledge Soundness). *The KZG polynomial commitment scheme over $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e)$ is evaluation-binding and knowledge-sound under the q -SDH (or q -DLOG) assumption. Specifically, given a valid opening proof $\pi = g^{w(\tau)}$ for commitment $C = g^f(\tau)$ at point z with value v , there exists a PPT extractor that outputs the unique polynomial $f(X) \in \mathbb{F}_p^{<s}[X]$ satisfying $f(z) = v$.*

Assumption 3 (Discrete Logarithm Hardness). *The discrete logarithm problem in $\mathbb{G}_1$ is computationally intractable. Given $g \in \mathbb{G}_1$ and $Y = g^y$, no PPT adversary can recover $y \in \mathbb{F}_p$ with non-negligible advantage.*

Theorem 1 (Correctness). *If the Data Owner, Cloud Server, and Verifier execute Algorithms 2, 3, 4, and 5 honestly, then Verify outputs accept with probability 1.*

Proof. Fix an honest execution for challenge chal = $(J, \{v_j\}_{j \in J}, \{r_i\}_{i=1}^m, \text{nonce})$. For each file $i \in [m]$, the prover computes $f_i(X) = \sum_{j \in J} v_j f_{i,j}(X)$ and $y_i = f_i(r_i)$. Since $f_i(r_i) - y_i = 0$, the quotient polynomial $w_i(X) = \frac{f_i(X) - y_i}{X - r_i}$ is well-defined with $\deg(w_i) \leq s - 2$. The global quotient $q(X) = \sum_{i=1}^m w_i(X)$ and folded polynomial $Q(X) =$

Algorithm 5 Verify**Require:** chal, proof π , roots $\{\Gamma_i\}_{i=1}^m, \{v_j\}$ **Ensure:** accept/reject

```

1: for  $i = 1$  to  $m$  do
2:   Check Merkle tree correctness with  $(\Gamma_i, \text{Auth}_i)$  for all
   chunks in  $J$ 
3:    $\hat{b}_i \leftarrow \prod_{j \in J} \sigma_{i,j}^{v_j}$ 
4:   if  $\hat{b}_i \neq b_i$  then
5:     return reject
6:   end if
7: end for
8:  $A \leftarrow \prod_{i=1}^m (b_i \cdot R_i \cdot g^{-\tilde{y}_i})$ 
9: if  $e(A \cdot C_{qr}, h) \neq e(C_q, h^\tau)$  then
10:  return reject
11: end if
12: for  $i = 1$  to  $m$  do
13:    $\alpha_i \leftarrow H_2(\text{chal}, \{\Gamma_i\}, \{b_i, \tilde{y}_i, R_i\}, C_q, C_{qr}, i)$ 
14: end for
15:  $C_Q \leftarrow C_q \cdot \prod_{i=1}^m b_i^{\alpha_i}$ 
16:  $z \leftarrow H_2(\text{chal}, \{\Gamma_i\}, \{b_i, \tilde{y}_i, R_i\}, C_q, C_{qr}, C_Q)$ 
17: if  $e(C_Q \cdot R_Q \cdot g^{-\tilde{V}_Q} \cdot \pi_{\text{batch}}^z, h) \neq e(\pi_{\text{batch}}, h^\tau)$  then
18:  return reject
19: end if
20: return accept

```

$q(X) + \sum_{i=1}^m \alpha_i f_i(X)$ both satisfy $\deg \leq s - 1$, ensuring all commitments are valid under the SRS.

The Merkle tree authentication can be checked with tree root Γ_i . Further, the tag recombination checks pass due to the adoption of homomorphic hash: $\hat{b}_i = \prod_{j \in J} \sigma_{i,j}^{v_j} = b_i = g^{f_i(\tau)}$.

For the quotient-consistency check, observe that for each i :

$$b_i \cdot R_i \cdot g^{-\tilde{y}_i} = g^{f_i(\tau)} \cdot g^{\mu_i} \cdot g^{-(y_i + \mu_i)} = g^{f_i(\tau) - y_i} = g^{(\tau - r_i)w_i(\tau)}.$$

Thus, $A := \prod_{i=1}^m (b_i R_i g^{-\tilde{y}_i}) = g^{\tau \sum_i w_i(\tau) - \sum_i r_i w_i(\tau)}$. By definition, $C_q = g^{q(\tau)}$ and $C_{qr} = g^{\sum_i r_i w_i(\tau)}$. Hence,

$$A \cdot C_{qr} = g^{\tau q(\tau)} = C_q^\tau.$$

Applying bilinearity yields $e(A \cdot C_{qr}, h) = e(C_q, h^\tau)$, satisfying the first pairing check.

Since the transcript is identical for both parties, the verifier derives identical folding coefficients $\alpha_i = H_2(\dots)$ and evaluation point $z = H_2(\dots)$. The folded commitment satisfies $C_Q = C_q \cdot \prod_i b_i^{\alpha_i} = g^{Q(\tau)}$. Finally,

$$\begin{aligned}
C_Q \cdot R_Q \cdot g^{-\tilde{V}_Q} \cdot \pi_{\text{batch}}^z &= g^{Q(\tau)} \cdot g^\tau \cdot g^{-(V_Q + \tau)} \cdot g^{z \frac{Q(\tau) - V_Q}{\tau - z}} \\
&= g^{Q(\tau) - V_Q} \cdot g^{\frac{z(Q(\tau) - V_Q)}{\tau - z}} \\
&= g^{(Q(\tau) - V_Q)(1 + \frac{z}{\tau - z})} \\
&= g^{(Q(\tau) - V_Q) \frac{\tau}{\tau - z}} \\
&= \left(g^{\frac{Q(\tau) - V_Q}{\tau - z}} \right)^\tau \\
&= \pi_{\text{batch}}^\tau.
\end{aligned}$$

By bilinearity, $e(C_Q \cdot R_Q \cdot g^{-\tilde{V}_Q} \cdot \pi_{\text{batch}}^z, h) = e(\pi_{\text{batch}}, h^\tau)$. All verification steps succeed deterministically, hence Verify outputs *accept*. $\square$

Theorem 2 (Soundness). *Let $\mathcal{A}$ be a PPT adversary controlling the Cloud Server. Under Assumptions 1–2 and modeling H_2 as a random oracle, if $\mathcal{A}$ outputs a proof π that causes Verify to accept for a challenge involving corrupted or unavailable data, then $\mathcal{A}$ succeeds with probability at most $\text{negl}(\lambda)$.*

Proof. Assume $\mathcal{A}$ produces an accepting proof $\pi = (\{\Gamma_i, \text{Auth}_i, b_i, \tilde{y}_i, R_i\}_{i=1}^m, C_q, C_{qr}, \tilde{V}_Q, R_Q, \pi_{\text{batch}})$. Acceptance implies three sequential constraints:

1. Authenticated Tag Binding. The verifier accepts the Merkle tree proofs and verifies $\hat{b}_i = \prod_{(j, \sigma_{i,j}) \in T_i} \sigma_{i,j}^{v_j} = b_i$. By Assumption 1, except with negligible probability, no adversary can forge an authentication path or find a collision in H_1 . Thus, each accepted b_i is cryptographically bound to the legitimate challenged tags $\{\sigma_{i,j}\}_{j \in J}$, and consequently to the polynomial $f_i(X) = \sum_{j \in J} v_j f_{i,j}(X)$ via $b_i = g^{f_i(\tau)}$.

2. Quotient-Consistency Enforcement. Since $R_i = g^{\mu_i}$ for a unique $\mu_i \in \mathbb{F}_p$, define $y_i := \tilde{y}_i - \mu_i$ and $w_i(X) := \frac{f_i(X) - y_i}{X - r_i}$. The accepted pairing equation $e(A \cdot C_{qr}, h) = e(C_q, h^\tau)$ implies $A \cdot C_{qr} = C_q^\tau$ in $\mathbb{G}_1$. Substituting $A = \prod_i g^{(\tau - r_i)w_i(\tau)}$ yields:

$$\begin{aligned}
g^{\tau \sum_i w_i(\tau) - \sum_i r_i w_i(\tau)} \cdot C_{qr} &= C_q^\tau \\
\implies C_{qr} &= g^{\sum_i r_i w_i(\tau)} \text{ and } C_q = g^{\sum_i w_i(\tau)}.
\end{aligned}$$

Thus, (C_q, C_{qr}) is algebraically bound to a unique global quotient polynomial $q(X) = \sum_i w_i(X)$ that satisfies the quotient relation for each file.

3. Fiat-Shamir Transcript Ordering & Folded Opening. The folding coefficients α_i are derived via H_2 after (C_q, C_{qr}) are fixed in the transcript. In the random oracle model, $\mathcal{A}$ cannot adaptively choose a malicious $q(X)$ as a function of $\{\alpha_i\}$ to cancel errors in the folded term $Q(X) = q(X) + \sum_i \alpha_i f_i(X)$.

The final pairing check $e(C_Q \cdot R_Q \cdot g^{-\tilde{V}_Q} \cdot \pi_{\text{batch}}^z, h) = e(\pi_{\text{batch}}, h^\tau)$ implies $C_Q \cdot R_Q \cdot g^{-\tilde{V}_Q} \cdot \pi_{\text{batch}}^z = \pi_{\text{batch}}^\tau$, which is exactly the KZG opening verification for commitment C_Q at point z with value $\tilde{V}_Q$. By Assumption 2 (knowledge soundness), there exists a PPT extractor that recovers a unique polynomial $\hat{F}(X) \in \mathbb{F}_p^{<s}[X]$ consistent with this opening. Since $C_Q = C_q \cdot \prod_i b_i^{\alpha_i}$, we must have $\hat{F}(X) = q(X) + \sum_i \alpha_i f_i(X)$.

If $\mathcal{A}$ substituted corrupted chunks, the corresponding $f_i(X)$ would deviate from the authentic encoding. This deviation propagates to $q(X)$ and $\hat{F}(X)$, causing the opening check to fail at the random point z with probability at least $1 - s/p$ by the Schwartz-Zippel lemma. Therefore, any accepting forgery implies a break of H_1 collision resistance, KZG knowledge soundness, or the random oracle property of H_2 , each occurring with negligible probability. $\square$

Theorem 3 (Privacy against Honest-but-Curious Verifier). *Under Assumption 3, the verifier cannot recover the challenged evaluations $\{y_i\}_{i=1}^m$ or the underlying data sectors from the audit transcript and proof π with non-negligible advantage.*

Proof. The proof transcript (in Algorithm 4) contains $(\tilde{y}_i, R_i)$ for each file i , and $(\tilde{V}_Q, R_Q)$ for the folded opening. By construction, $\tilde{y}_i = y_i + \mu_i$ where $\mu_i \xleftarrow{\$} \mathbb{F}_p$ is freshly sampled per audit round. Since μ_i is uniformly distributed and independent of y_i , $\tilde{y}_i$ is statistically indistinguishable from a uniform field element (one-time pad property). The commitment $R_i = g^{\mu_i}$ computationally hides μ_i under Assumption 3. Recovering μ_i from R_i is equivalent to solving the discrete logarithm problem in $\mathbb{G}_1$, which is infeasible for PPT adversaries. The same argument applies to $(\tilde{V}_Q, R_Q)$ with fresh mask η .

Since all masking values are refreshed independently in every audit round, cross-round correlation attacks are prevented. The remaining transcript elements (challenges, commitments C_q, C_{qr}, C_Q , and π_{batch}) reveal only algebraic relations bound to the masked values. Consequently, the verifier's view of $\{y_i\}$ or the raw sectors $\{m_{i,j,k}\}$ is computationally indistinguishable from random, establishing privacy against an honest-but-curious verifier. $\square$

C. Complexity Analysis

a) Computation Complexity. The computational overhead of the scheme is dominated by group exponentiations and bilinear pairings. Let E, P denote the time overhead of exponentiation and bilinear pairing, respectively. During proof generation (Algorithm 4), the prover processes each of the m files independently. For every file i , reconstructing the aggregated tag b_i from c challenged sectors requires c exponentiations, while generating the per-file privacy masking commitment R_i and its associated SRS-based evaluation consumes 2 exponentiations. This yields a per-file cost of $(c+2)E$, scaling to $m(c+2)E$ across the batch. Additionally, the prover computes the global quotient commitments C_q and C_{qr} , the global masking commitment R_Q , and the final batch opening proof π_{batch} , each requiring exactly one exponentiation via the SRS. Consequently, the prover's total exponentiation complexity is precisely $m(c+2)E + 4E$. During verification (Algorithm 5), the verifier first authenticates the challenged tags by recomputing the aggregated tag $\hat{b}_i$ for each file (c exponentiations) and verifying one auxiliary commitment, contributing $m(c+1)E$. The core cryptographic validation relies on two constant-size pairing checks: the first enforces quotient consistency via $e(A \cdot C_{qr}, h) \stackrel{?}{=} e(C_q, h^\tau)$, and the second verifies the folded polynomial opening via $e(C_Q R_Q g^{-\tilde{V}_Q} \pi_{\text{batch}}^z, h) \stackrel{?}{=} e(\pi_{\text{batch}}, h^\tau)$. Each check consumes exactly two pairings, resulting in a fixed verifier pairing complexity of $4P$, which remains strictly independent of both the batch size m and the number of challenged chunks c . (As noted in Remark 1, these two checks can be merged via random linear combination to reduce the pairing cost to $2P$ in practice.)

b) Communication Complexity. The communication overhead is determined by the byte size of the audit proof π transmitted from the cloud server to the verifier. Let $|G_1|$ denote the size of a group element in $\mathbb{G}_1$ and $|p|$ denote the size of a field element in $\mathbb{F}_p$. Per file, the proof payload includes c challenged chunk tags, one aggregated tag b_i , one masking

commitment R_i , and one masked field evaluation $\tilde{y}_i$, contributing $(c+2)|G_1| + |p|$ per file. Furthermore, each file requires a compact Tag-IMHT multi-proof authentication path consisting of $\nu_{\text{MP}}(c, n)$ hash digests, adding $\nu_{\text{MP}}(c, n)|h|$ to the per-file transmission. Across m files, the linear communication cost is therefore $m((c+2)|G_1| + |p| + \nu_{\text{MP}}(c, n)|h|)$. The cross-file batch aggregation layer introduces a fixed cryptographic overhead that is invariant to m : two quotient commitments (C_q, C_{qr}) , one folded commitment (C_Q) , one global masking commitment (R_Q) , one batch opening proof (π_{batch}) , and one globally masked evaluation $(\tilde{V}_Q)$. This constant footprint adds exactly $4|G_1| + 2|p|$ to the total bandwidth. By replacing c independent Merkle authentication paths with a single multi-proof per file, the protocol significantly compresses the authenticated evidence payload, ensuring that the total communication scales linearly with m and c while the batch-aggregation cryptographic terms remain strictly constant.

c) Optional merged verification. If the two pairing checks are merged via random linear combination, the verifier pairing cost can be reduced from $4P$ to $2P$ in practice (with the standard batching soundness caveat in the random oracle model).

D. Dynamic File Management

The proposed batch-PDP scheme natively supports dynamic file operations through its modular Merkle tree structure and independent per-file polynomial commitments. File addition and deletion can be performed efficiently without regenerating public parameters, or recomputing tags for unrelated files. The batch verification framework (Algorithm 5) seamlessly accommodates dynamic file sets without protocol modification.

a) File Addition. When a Data Owner (DO) uploads a new file F_{m+1} to the Cloud Server (CS), the following fields and states must be updated across the protocol lifecycle:

- **Store Algorithm:** The DO increments the batch size $m \leftarrow m + 1$. For each chunk $j \in [n']$ of the new file, the DO computes the polynomial encoding $f_{m+1,j}(X)$, generates homomorphic tags $\sigma_{m+1,j} = g^{f_{m+1,j}(\tau)}$, and constructs leaf nodes $L_{m+1,j} = H_1((m+1) \| j \| \sigma_{m+1,j})$. A new Merkle tree is built to yield the root Γ_{m+1} . The DO securely stores Γ_{m+1} and shares it with the Verifier (V). The CS receives $\{F_{m+1,j}\}$, $\{\sigma_{m+1,j}\}$, and the Merkle authentication nodes.
- **ProofGen Algorithm:** The loop bound in Algorithm 4 extends to $i = 1$ to $m + 1$. The newly computed per-file fields $\{f_{m+1}(X), b_{m+1}, y_{m+1}, \tilde{y}_{m+1}, R_{m+1}, g_{m+1}(X)\}$ are incorporated into the global aggregation layer. Specifically, the global quotient polynomial updates as $q_{\text{new}}(X) \leftarrow q(X) + g_{m+1}(X)$, and the commitments become $C_q \leftarrow C_q \cdot g^{g_{m+1}(\tau)}$, $C_{qr} \leftarrow C_{qr} \cdot (g^{g_{m+1}(\tau)})^{r_{m+1}}$. The Fiat-Shamir transcript naturally absorbs Γ_{m+1} and the new per-file terms. Consequently, the folding coefficients $\{\alpha_i\}_{i=1}^{m+1}$ are freshly derived via H_2 over the expanded transcript. The folded polynomial updates to $Q(X) \leftarrow q_{\text{new}}(X) + \sum_{i=1}^{m+1} \alpha_i f_i(X)$, and the folded commitment becomes $C_Q \leftarrow C_Q \cdot \prod_{i=1}^{m+1} b_i^{\alpha_i}$. The evaluation point z is recomputed as

Table II: Communication and Computation Overhead (Updated This Work)

Category	Phase	Ours-SP (single-path)	Ours-MP (multi-proof)
Communication Overhead	Store	$mns p + mn G_1 + m h $	$mns p + mn G_1 + m h $
	Chal	$c p + c\log_2 n + m p $	$c p + c\log_2 n + m p $
	ProofGen	$m((c+2) G_1 + p + c\log_2 n h) + 4 G_1 + 2 p $	$m((c+2) G_1 + p + \nu_{MP}(c, n) h) + 4 G_1 + 2 p $
Computation Overhead	Store	$mnE + m(2n-1)H + O(mns)F$	$mnE + m(2n-1)H + O(mns)F$
	ProofGen	$m((c+2)E + (c-1)M + c\log_2 nH) + 4E + O(mcs)F$	$m((c+2)E + (c-1)M + \gamma_{MP}(c, n)H) + 4E + O(mcs)F$
	ProofVeri	$4P + m((c+1)E + cM + c\log_2 nH) + O(m)H$	$4P + m((c+1)E + cM + \tilde{\gamma}_{MP}(c, n)H) + O(m)H$

Table III: Compact TC25-Style Comparison (Updated Verifier)

Category	Phase	YuTC25 (Table II, Our)	ZhangTPDS23 (Batch)	Ours-MP (Updated)
Communication	Store	$mn p + m G_1 + h $	$mN(Copy + n G_1)^{\dagger}$	$mns p + mn G_1 + m h $
	Chal	$2 p + \log_2 c$	$m(c\log_2 n + p + p) + 2 p ^{\dagger}$	$c p + c\log_2 n + m p $
	ProofGen	$(n+1) p + c\log_2 m h + (c+1) G_1 $	$3 G_1 + p $ (Table VIII, multiple files)	$m((c+2) G_1 + p + \nu_{MP}(c, n) h) + 4 G_1 + 2 p $
Computation	Store	E	$2mNnE + mNnM^{\ddagger}$	$mnE + m(2n-1)H + O(mns)F$
	ProofGen	E	$(mc + 2s - 5)M + (mc + m + 2s - 2)E$	$m((c+2)E + (c-1)M + \gamma_{MP}(c, n)H) + 4E + O(mcs)F$
	ProofVeri	$(n+c+4)E + (n+c+1)M$	$4P + (Nmc + Nm + 3)E + (Nmc + 3N - 2)M$	$4P + m((c+1)E + cM + \tilde{\gamma}_{MP}(c, n)H) + O(m)H$

$\dagger$ derived from ZhangTPDS23 protocol message format in batch mode.

$\ddagger$ scaled from ZhangTPDS23 Table V (TagGen) to m files.

$z \leftarrow H_2(\cdot, \{\Gamma_i\}_{i=1}^{m+1}, \{b_i, \tilde{y}_i, R_i\}_{i=1}^{m+1}, C_q, C_{qr}, C_Q)$. The folded evaluation updates to $V_Q \leftarrow Q(z)$, and the batch opening proof π_{batch} is regenerated using the new $Q(\tau)$ and V_Q .

b) File Deletion.: To securely remove an existing file $F_{i_{del}}$ ($1 \leq i_{del} \leq m$), the system performs lightweight state pruning. The CS permanently erases $\{F_{i_{del},j}\}$, $\{\sigma_{i_{del},j}\}$, and the associated Merkle tree. The DO and V remove $\Gamma_{i_{del}}$ from their authenticated root sets. The active file count decrements to $m \leftarrow m-1$. To avoid recomputing tags and Merkle roots for surviving files, the scheme retains original file indices (sparse indexing). Consequently, deletion incurs zero cryptographic recomputation for remaining files. The fields $\{\sigma_{k,j}\}$ and Γ_k for $k \neq i_{del}$ remain valid and unchanged. In ProofGen, the values or variables $q(X), C_q, C_{qr}, Q(X), C_Q, z, V_Q$ are updated, according to Algorithm 4.

c) Complexity Analysis of Dynamic Operations.: Dynamic file lifecycle management requires synchronized updates across the Merkle authentication layer and the polynomial commitment layer. The computational overhead for updating the specified fields $\{\Gamma, q(X), C_q, C_{qr}, Q(X), C_Q, z, V_Q\}$ during addition and deletion is analyzed as follows:

E. Comparison Paragraph (for related-work discussion)

Compared with YuTC25, this scheme keeps Tag-IMHT indexed authentication but adds cross-file batch folding and single-point opening. Compared with ZhangTPDS23, this scheme preserves batch-auditing efficiency while explicitly authenticating challenged tags/indices via Tag-IMHT. The remaining linear factor mainly comes from authenticated evidence, which is reduced by multi-proof.

REFERENCES

- [1] Luo, Z., Jia, Y., Gracia, A.V.O. and Kate, A. Cauchyproofs: Batch-Updateable Vector Commitment with Easy Aggregation and Application to Stateless Blockchains. In *S&P*, 2025, pp. 1947-1963.
- [2] Kate, A., Zaverucha, G.M. and Goldberg, I. Constant-size commitments to polynomials and their applications. In *Asiacrypt*, 2010, pp. 177-194.

- [3] Boneh, D., Drake, J., Fisch, B. & Gabizon, A. Efficient polynomial commitment schemes for multiple points and polynomials. *Cryptology ePrint Archive*. (2020)
- [4] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge," *IACR Cryptology ePrint Archive*, Report 2019/953, 2019. <https://eprint.iacr.org/2019/953>

Table IV: Theoretical Overhead Comparison (TC25-style, compact)

Category	Phase	YuTC25	ZhangTPDS23 (Batch)	Ours-MP (This Work)
Communication	Store	$mn p + m G_1 + h $	$mN(Copy + n G_1)^{\dagger}$	$mns p + mn G_1 + m h $
	Chal	$2 p + \log_2 c$	$m(c \log_2 n + p + p) + 2 p ^{\dagger}$	$c p + c \log_2 n + m p $
	ProofGen	$(n+1) p + c \log_2 m h + (c+1) G_1 $	$3 G_1 + p $ (Table VIII, multi-files)	$m((c+2) G_1 + p + \nu_{MP}(c, n) h) + 3 G_1 + p $
Computation	Store	E	$2mNnE + mNnM^{\ddagger}$	$mnE + m(2n-1)H + O(mns)F$
	ProofGen	E	$(mc + 2s - 5)M + (mc + m + 2s - 2)E$	$m((c+2)E + (c-1)M + \gamma_{MP}(c, n)H) + 3E + O(mcs)F$
	ProofVeri	$(n+c+4)E + (n+c+1)M$	$4P + (Nmc + Nm + 3)E + (Nmc + 3N - 2)M$	$2P + m((c+1)E + cM + \tilde{\gamma}_{MP}(c, n)H) + O(m)H$

$\dagger$ communication terms are derived from ZhangTPDS23 protocol messages in batch mode.

$\ddagger$ scaled from ZhangTPDS23 Table V (TagGen) to m files.